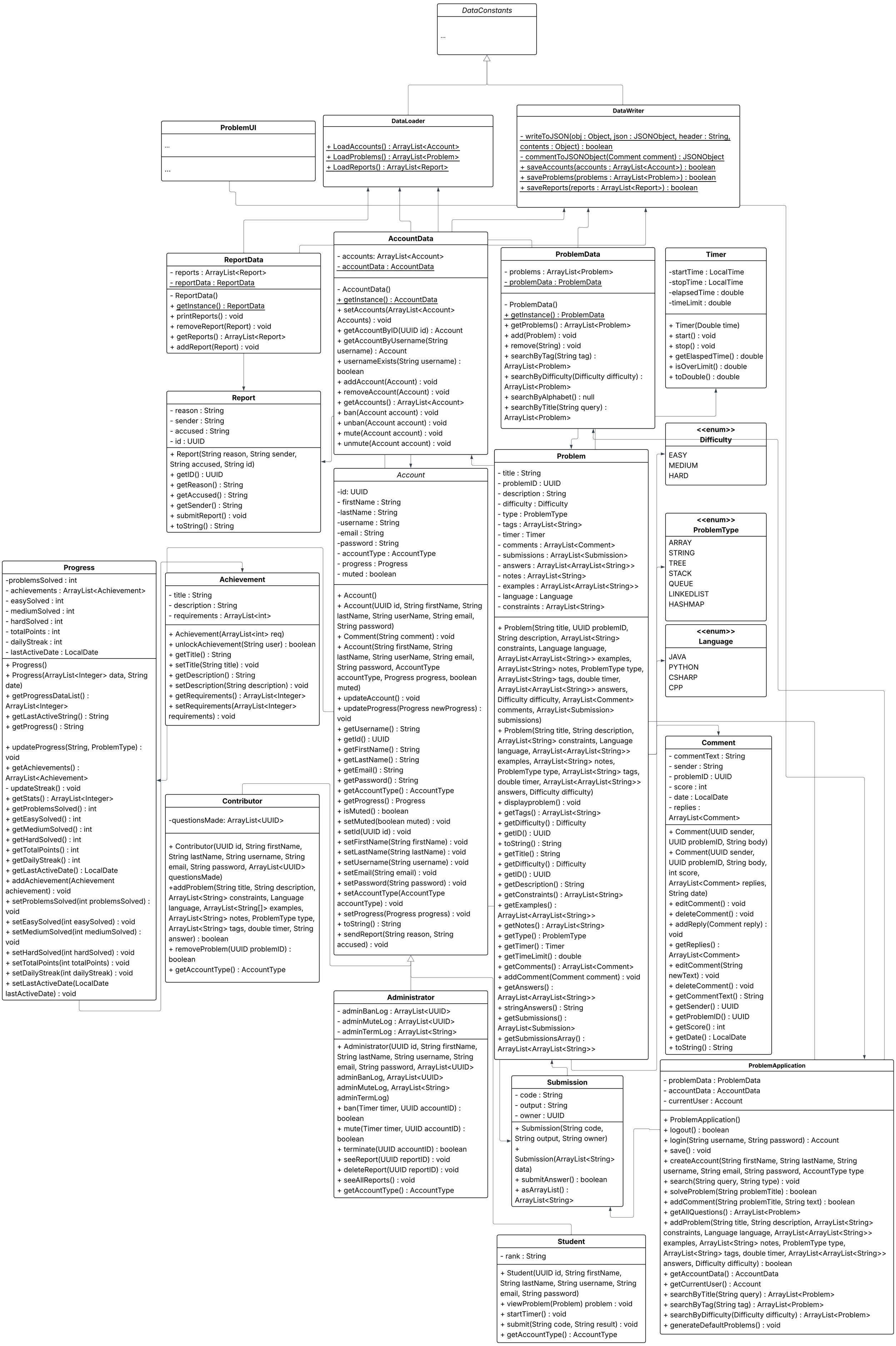
DataConstants
...
ProblemUI
...
...
DataLoader
+ LoadAccounts() : ArrayList<Account>
+ LoadProblems() : ArrayList<Problem>
+ LoadReports() : ArrayList<Report>
DataWriter
- writeToJSON(obj : Object, json : JSONObject, header : String, contents : Object) : boolean
- commentToJSONObject(Comment comment) : JSONObject
+ saveAccounts(accounts : ArrayList<Account>) : boolean
+ saveProblems(problems : ArrayList<Problem>) : boolean
+ saveReports(reports : ArrayList<Report>) : boolean
AccountData
- accounts: ArrayList<Account>
- accountData : AccountData
- AccountData()
+ getInstance() : AccountData
+ setAccounts(ArrayList<Account> Accounts) : void
+ getAccountByID(UUID id) : Account
+ getAccountByUsername(String username) : Account
+ usernameExists(String username) : boolean
+ addAccount(Account) : void
+ removeAccount(Account) : void
+ getAccounts() : ArrayList<Account>
+ ban(Account account) : void
+ unban(Account account) : void
+ mute(Account account) : void
+ unmute(Account account) : void
ReportData
- reports : ArrayList<Report>
- reportData : ReportData
- ReportData()
+ getInstance() : ReportData
+ printReports() : void
+ removeReport(Report) : void
+ getReports() : ArrayList<Report>
+ addReport(Report) : void
ProblemData
- problems : ArrayList<Problem>
- problemData : ProblemData
- ProblemData()
+ getInstance() : ProblemData
+ getProblems() : ArrayList<Problem>
+ add(Problem) : void
+ remove(String) : void
+ searchByTag(String tag) : ArrayList<Problem>
+ searchByDifficulty(Difficulty difficulty) : ArrayList<Problem>
+ searchByAlphabet() : null
+ searchByTitle(String query) : ArrayList<Problem>
Timer
-startTime : LocalTime
-stopTime : LocalTime
-elapsedTime : double
-timeLimit : double
+ Timer(Double time)
+ start() : void
+ stop() : void
+ getElaspedTime() : double
+ isOverLimit() : double
+ toDouble() : double
Report
- reason : String
- sender : String
- accused : String
- id : UUID
+ Report(String reason, String sender, String accused, String id)
+ getID() : UUID
+ getReason() : String
+ getAccused() : String
+ getSender() : String
+ submitReport() : void
+ toString() : String
<<enum>> Difficulty
EASY
MEDIUM
HARD
Account
-id: UUID
- firstName : String
-lastName : String
-username : String
-email : String
-password : String
- accountType : AccountType
- progress : Progress
- muted : boolean
+ Account()
+ Account(UUID id, String firstName, String lastName, String userName, String email, String password)
+ Comment(String comment) : void
+ Account(String firstName, String lastName, String userName, String email, String password, AccountType accountType, Progress progress, boolean muted)
+ updateAccount() : void
+ updateProgress(Progress newProgress) : void
+ getUsername() : String
+ getId() : UUID
+ getFirstName() : String
+ getLastName() : String
+ getEmail() : String
+ getPassword() : String
+ getAccountType() : AccountType
+ getProgress() : Progress
+ isMuted() : boolean
+ setMuted(boolean muted) : void
+ setId(UUID id) : void
+ setFirstName(String firstName) : void
+ setLastName(String lastName) : void
+ setUsername(String username) : void
+ setEmail(String email) : void
+ setPassword(String password) : void
+ setAccountType(AccountType accountType) : void
+ setProgress(Progress progress) : void
+ toString() : String
+ sendReport(String reason, String accused) : void
Problem
- title : String
- problemID : UUID
- description : String
- difficulty : Difficulty
- type : ProblemType
- tags : ArrayList<String>
- timer : Timer
- comments : ArrayList<Comment>
- submissions : ArrayList<Submission>
- answers : ArrayList<ArrayList<String>>
- notes : ArrayList<String>
- examples : ArrayList<ArrayList<String>>
- language : Language
- constraints : ArrayList<String>
+ Problem(String title, UUID problemID, String description, ArrayList<String> constraints, Language language, ArrayList<ArrayList<String>> examples, ArrayList<String> notes, ProblemType type, ArrayList<String> tags, double timer, ArrayList<ArrayList<String>> answers, Difficulty difficulty, ArrayList<Comment> comments, ArrayList<Submission> submissions)
+ Problem(String title, String description, ArrayList<String> constraints, Language language, ArrayList<ArrayList<String>> examples, ArrayList<String> notes, ProblemType type, ArrayList<String> tags, double timer, ArrayList<ArrayList<String>> answers, Difficulty difficulty)
+ displayproblem() : void
+ getTags() : ArrayList<String>
+ getDifficulty() : Difficulty
+ getID() : UUID
+ toString() : String
+ getTitle() : String
+ getDifficulty() : Difficulty
+ getID() : UUID
+ getDescription() : String
+ getConstraints() : ArrayList<String>
+ getExamples() : ArrayList<ArrayList<String>>
+ getNotes() : ArrayList<String>
+ getType() : ProblemType
+ getTimer() : Timer
+ getTimeLimit() : double
+ getComments() : ArrayList<Comment>
+ addComment(Comment comment) : void
+ getAnswers() : ArrayList<ArrayList<String>>
+ stringAnswers() : String
+ getSubmissions() : ArrayList<Submission>
+ getSubmissionsArray() : ArrayList<ArrayList<String>>
<<enum>> ProblemType
ARRAY
STRING
TREE
STACK
QUEUE
LINKEDLIST
HASHMAP
<<enum>> Language
JAVA
PYTHON
CSHARP
CPP
Progress
-problemsSolved : int
- achievements : ArrayList<Achievement>
- easySolved : int
- mediumSolved : int
- hardSolved : int
- totalPoints : int
- dailyStreak : int
- lastActiveDate : LocalDate
+ Progress()
+ Progress(ArrayList<Integer> data, String date)
+ getProgressDataList() : ArrayList<Integer>
+ getLastActiveString() : String
+ getProgress() : String
+ updateProgress(String, ProblemType) : void
+ getAchievements() : ArrayList<Achievement>
- updateStreak() : void
+ getStats() : ArrayList<Integer>
+ getProblemsSolved() : int
+ getEasySolved() : int
+ getMediumSolved() : int
+ getHardSolved() : int
+ getTotalPoints() : int
+ getDailyStreak() : int
+ getLastActiveDate() : LocalDate
+ addAchievement(Achievement achievement) : void
+ setProblemsSolved(int problemsSolved) : void
+ setEasySolved(int easySolved) : void
+ setMediumSolved(int mediumSolved) : void
+ setHardSolved(int hardSolved) : void
+ setTotalPoints(int totalPoints) : void
+ setDailyStreak(int dailyStreak) : void
+ setLastActiveDate(LocalDate lastActiveDate) : void
Achievement
- title : String
- description : String
- requirements : ArrayList<int>
+ Achievement(ArrayList<int> req)
+ unlockAchievement(String user) : boolean
+ getTitle() : String
+ setTitle(String title) : void
+ getDescription() : String
+ setDescription(String description) : void
+ getRequirements() : ArrayList<Integer>
+ setRequirements(ArrayList<Integer> requirements) : void
Contributor
-questionsMade: ArrayList<UUID>
+ Contributor(UUID id, String firstName, String lastName, String username, String email, String password, ArrayList<UUID> questionsMade)
+addProblem(String title, String description, ArrayList<String> constraints, Language language, ArrayList<String[]> examples, ArrayList<String> notes, ProblemType type, ArrayList<String> tags, double timer, String answer) : boolean
+ removeProblem(UUID problemID) : boolean
+ getAccountType() : AccountType
Comment
- commentText : String
- sender : String
- problemID : UUID
- score : int
- date : LocalDate
- replies : ArrayList<Comment>
+ Comment(UUID sender, UUID problemID, String body)
+ Comment(UUID sender, UUID problemID, String body, int score, ArrayList<Comment> replies, String date)
+ editComment() : void
+ deleteComment() : void
+ addReply(Comment reply) : void
+ getReplies() : ArrayList<Comment>
+ editComment(String newText) : void
+ deleteComment() : void
+ getCommentText() : String
+ getSender() : UUID
+ getProblemID() : UUID
+ getScore() : int
+ getDate() : LocalDate
+ toString() : String
Administrator
- adminBanLog : ArrayList<UUID>
- adminMuteLog : ArrayList<UUID>
- adminTermLog : ArrayList<String>
+ Administrator(UUID id, String firstName, String lastName, String username, String email, String password, ArrayList<UUID> adminBanLog, ArrayList<UUID> adminMuteLog, ArrayList<String> adminTermLog)
+ ban(Timer timer, UUID accountID) : boolean
+ mute(Timer timer, UUID accountID) : boolean
+ terminate(UUID accountID) : boolean
+ seeReport(UUID reportID) : void
+ deleteReport(UUID reportID) : void
+ seeAllReports() : void
+ getAccountType() : AccountType
Submission
- code : String
- output : String
- owner : UUID
+ Submission(String code, String output, String owner)
+ Submission(ArrayList<String> data)
+ submitAnswer() : boolean
+ asArrayList() : ArrayList<String>
ProblemApplication
- problemData : ProblemData
- accountData : AccountData
- currentUser : Account
+ ProblemApplication()
+ logout() : boolean
+ login(String username, String password) : Account
+ save() : void
+ createAccount(String firstName, String lastName, String username, String email, String password, AccountType type
+ search(String query, String type) : void
+ solveProblem(String problemTitle) : boolean
+ addComment(String problemTitle, String text) : boolean
+ getAllQuestions() : ArrayList<Problem>
+ addProblem(String title, String description, ArrayList<String> constraints, Language language, ArrayList<ArrayList<String>> examples, ArrayList<String> notes, ProblemType type, ArrayList<String> tags, double timer, ArrayList<ArrayList<String>> answers, Difficulty difficulty) : boolean
+ getAccountData() : AccountData
+ getCurrentUser() : Account
+ searchByTitle(String query) : ArrayList<Problem>
+ searchByTag(String tag) : ArrayList<Problem>
+ searchByDifficulty(Difficulty difficulty) : ArrayList<Problem>
+ generateDefaultProblems() : void
Student
- rank : String
+ Student(UUID id, String firstName, String lastName, String username, String email, String password)
+ viewProblem(Problem) problem : void
+ startTimer() : void
+ submit(String code, String result) : void
+ getAccountType() : AccountType